Mastering Diffusion

Week 7 – Technical Components

Conditioning, Diffusion Distillation

Jaeik Kim

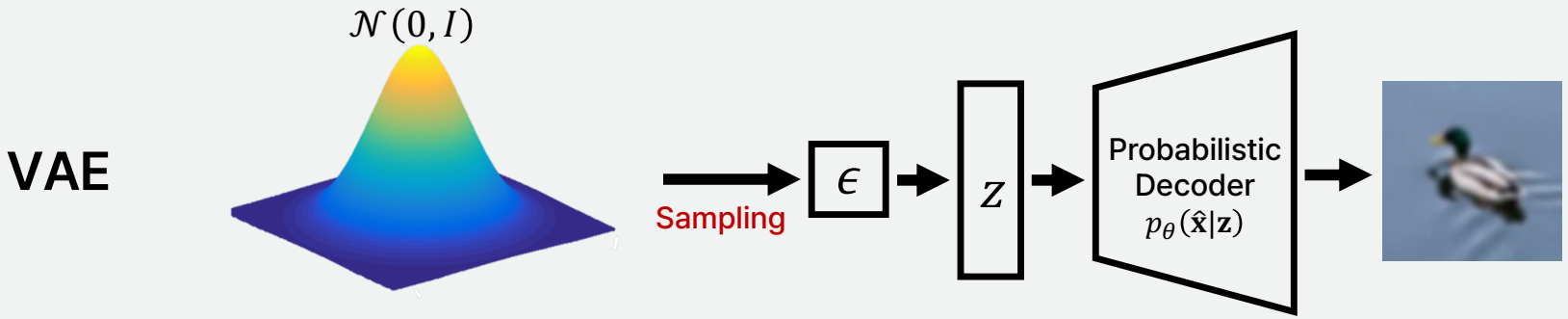
SNU AIDAS Lab

AIDAS Lab

Conditioning

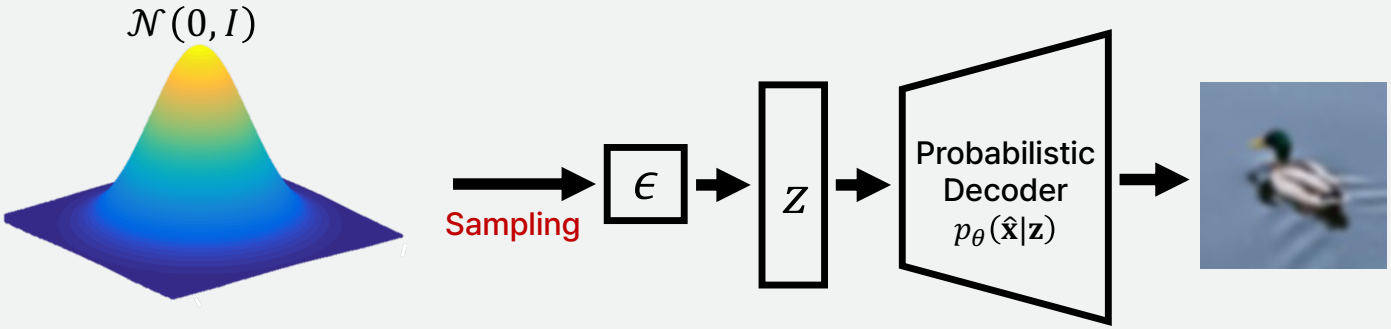
Up to the last session

VAE

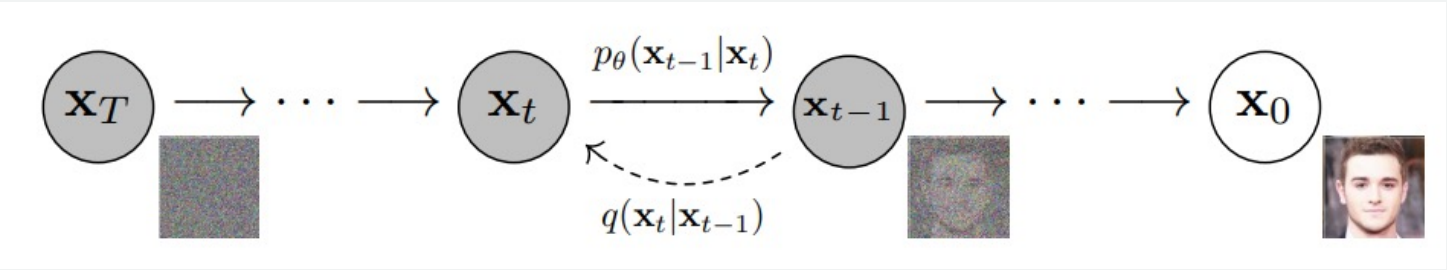


Up to the last session

VAE

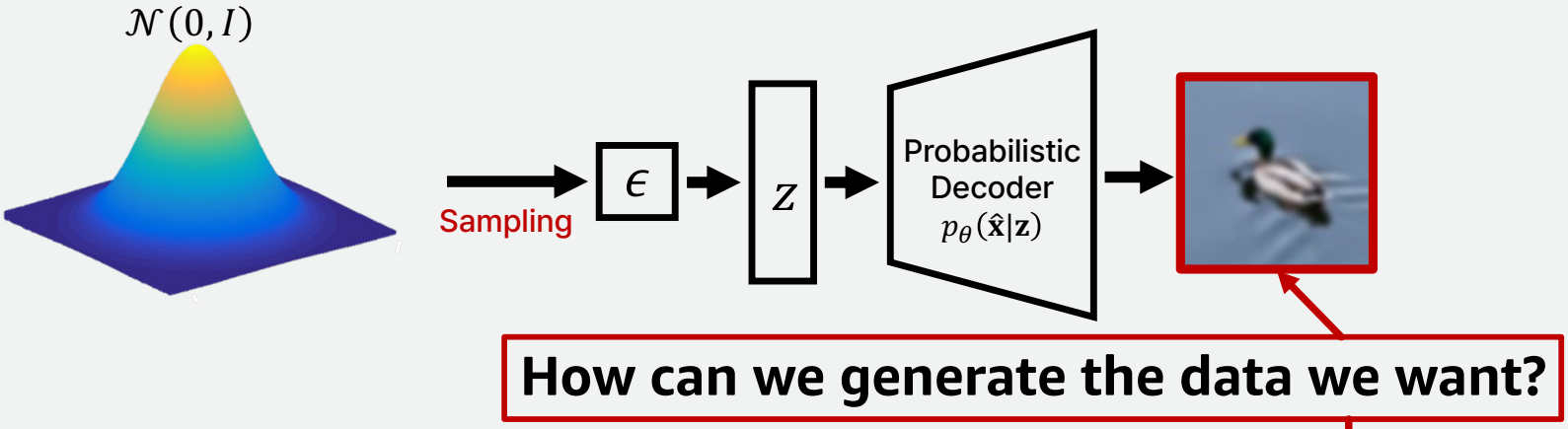


Diffusion



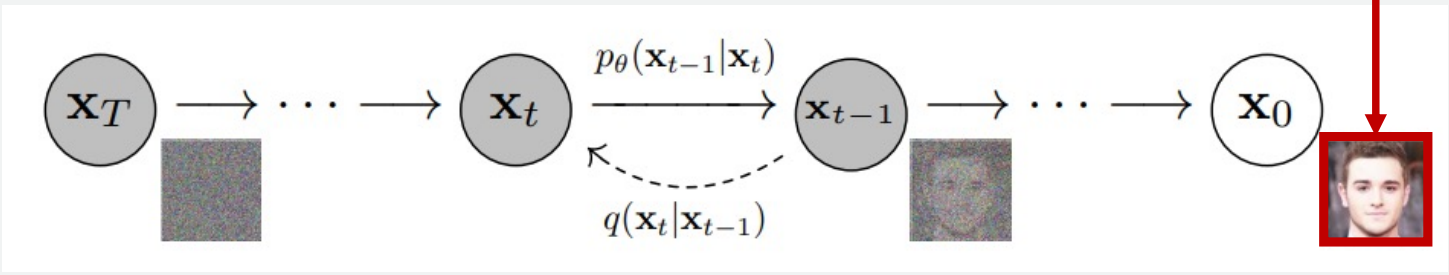
Up to the last session

VAE



How can we generate the data we want?

Diffusion



Adaptive Group Normalization

$$\mathrm{AdaGN}(h, y) = y_s \cdot \mathrm{GroupNorm}(h) + y_b,$$

$$where$$

y_s, y_b: projected timestep & class embedding

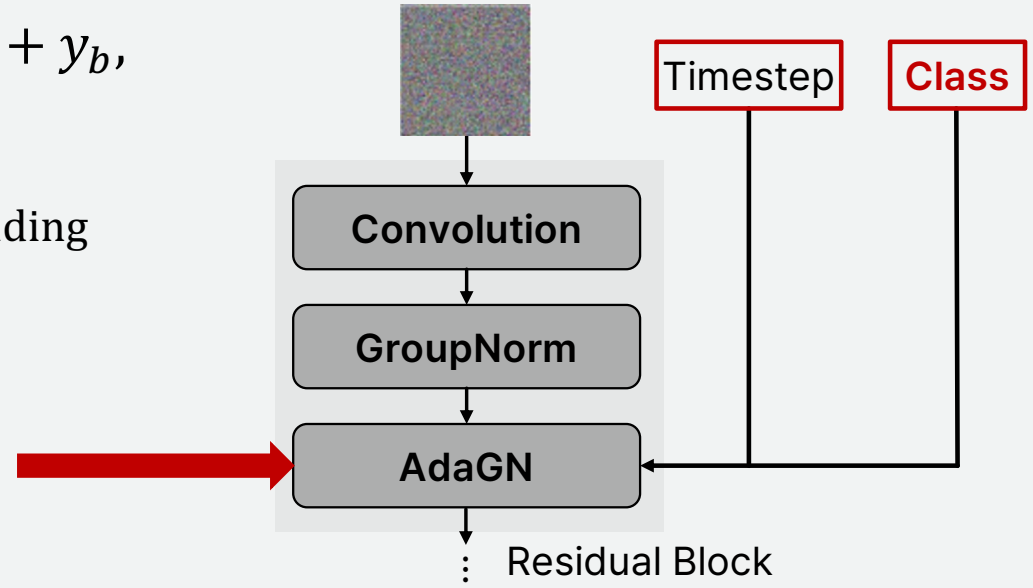
Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Adaptive Group Normalization

$$\mathrm{AdaGN}(h, y) = y_s \cdot \mathrm{GroupNorm}(h) + y_b,$$

$$where$$

y_s, y_b: projected timestep & class embedding



Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

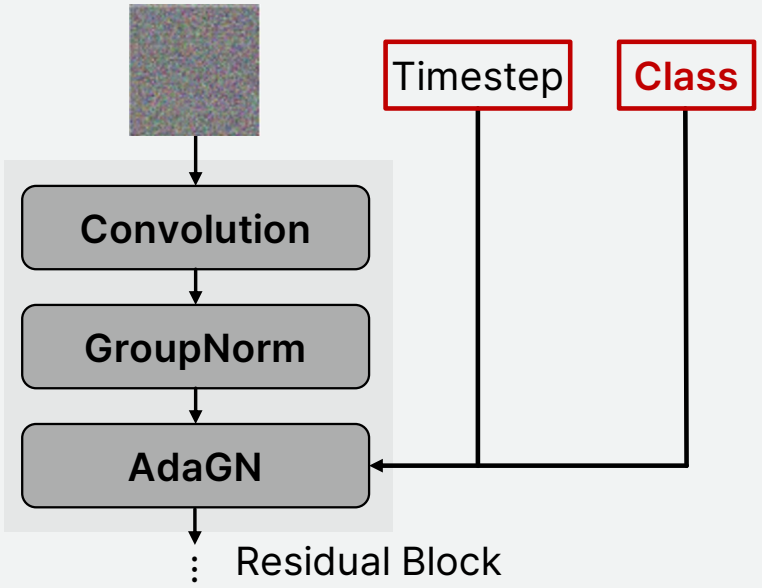
Adaptive Group Normalization

$$\mathrm{AdaGN}(h, y) = y_s \cdot \mathrm{GroupNorm}(h) + y_b,$$

$$where$$

y_s, y_b: projected timestep & class embedding

Operation	FID
AdaGN	13.06
Addition + GroupNorm	15.08



Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Unconditional denoising process: $p_\theta(x_t|x_{t+1})$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Unconditional denoising process: $p_\theta(x_t|x_{t+1})$

Conditional denoising process:

$$p_{\theta,\phi}(x_t|x_{t+1}, y)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Unconditional denoising process: $p_\theta(x_t|x_{t+1})$

Conditional denoising process:

$$p_{\theta,\phi}(x_t|x_{t+1}, y) = Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Unconditional denoising process: $p_\theta(x_t|x_{t+1})$

Conditional denoising process:

$$p_{\theta,\phi}(x_t|x_{t+1}, y) = Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Assume class-conditioned noising process $\hat{q}\ s.t.$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Assume class-conditioned noising process $\hat{q}\ s.t.$

$$\hat{q}(x_0) \coloneqq q(x_0)$$

$$\hat{q}(y|x_0) \coloneqq \text{Known labels per sample}$$

$$\hat{q}(x_{t+1}|x_t, y) \coloneqq q(x_{t+1}|x_t)$$

$$\hat{q}(x_{1:T}|x_0, y) \coloneqq \prod_{t=1} \hat{q}(x_t|x_{t-1}, y)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Assume class-conditioned noising process $\hat{q}$ $s.t.$

$$\hat{q}(x_0) \coloneqq q(x_0)$$

$$\hat{q}(y|x_0) \coloneqq \text{Known labels per sample}$$

$$\hat{q}(x_{t+1}|x_t, y) \coloneqq q(x_{t+1}|x_t)$$

$$\hat{q}(x_{1:T}|x_0, y) \coloneqq \prod_{t=1} \hat{q}(x_t|x_{t-1}, y)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy = \int_y \hat{q}(x_{t+1}|x_t, y)\, \hat{q}(y|x_t) dy \quad \cdots \text{ (By Bayes Rule)}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy = \int_y \hat{q}(x_{t+1}|x_t, y)\, \hat{q}(y|x_t) dy \quad \cdots \text{ (By Bayes Rule)}$$

$$= \int_y q(x_{t+1}|x_t)\, \hat{q}(y|x_t) dy$$

Assumption $\hat{\mathbf{q}}(\mathbf{x_{t+1}}|\mathbf{x_t}, \mathbf{y}) \coloneqq \mathbf{q}(\mathbf{x_{t+1}}|\mathbf{x_t})$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy = \int_y \hat{q}(x_{t+1}|x_t, y)\, \hat{q}(y|x_t) dy \quad \cdots \text{ (By Bayes Rule)}$$

$$= \int_y q(x_{t+1}|x_t)\, \hat{q}(y|x_t) dy = q(x_{t+1}|x_t) \int_y \hat{q}(y|x_t)\, dy$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy = \int_y \hat{q}(x_{t+1}|x_t, y)\, \hat{q}(y|x_t) dy \quad \cdots \text{(By Bayes Rule)}$$

$$= \int_y q(x_{t+1}|x_t)\, \hat{q}(y|x_t) dy = q(x_{t+1}|x_t) \int_y \hat{q}(y|x_t)\, dy$$

$$= q(x_{t+1}|x_t)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy = \int_y \hat{q}(x_{t+1}|x_t, y)\, \hat{q}(y|x_t) dy \quad \cdots \text{ (By Bayes Rule)}$$

$$= \int_y q(x_{t+1}|x_t)\, \hat{q}(y|x_t) dy = q(x_{t+1}|x_t) \int_y \hat{q}(y|x_t)\, dy$$

$$= q(x_{t+1}|x_t) = \hat{q}(x_{t+1}|x_t, y)$$

Assumption $\hat{\mathbf{q}}(\mathbf{x_{t+1}}|\mathbf{x_t}, \mathbf{y}) \coloneqq \mathbf{q}(\mathbf{x_{t+1}}|\mathbf{x_t})$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

$$\hat{q}(x_{t+1}|x_t) = \int_y \hat{q}(x_{t+1}, y|x_t)\, dy = \int_y \hat{q}(x_{t+1}|x_t, y)\, \hat{q}(y|x_t) dy \quad \cdots \text{(By Bayes Rule)}$$

$$= \int_y q(x_{t+1}|x_t)\, \hat{q}(y|x_t) dy = q(x_{t+1}|x_t) \int_y \hat{q}(y|x_t)\, dy$$

$$= q(x_{t+1}|x_t) = \hat{q}(x_{t+1}|x_t, y)$$

Noise process behaves exactly like q when not conditioned on y

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Similarly, for joint distribution:

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Similarly, for joint distribution:

$$\hat{q}(x_{1:T}|x_0) = \int_y \hat{q}(x_{1:T}, y|x_0)\, dy = \int_y \hat{q}(x_{1:T}|x_0, y)\, \hat{q}(y|x_0) dy$$

$$= \int_y \prod \hat{q}(x_t|x_{t-1}, y)\, \hat{q}(y|x_0) dy = \int_y \prod q(x_t|x_{t-1})\, \hat{q}(y|x_0)\, dy$$

$$= \prod q(x_t|x_{t-1}) \int_y \hat{q}(y|x_0)\, dy = \prod q(x_t|x_{t-1}) = q(x_{1:T}|x_0)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classification distribution does not depend on x_{t+1} :

$$\hat{q}(y|x_t, x_{t+1}) = \hat{q}(x_{t+1}|x_t, y)\frac{\hat{q}(y|x_t)}{\hat{q}(x_{t+1}|x_t)}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classification distribution does not depend on x_{t+1} :

$$\hat{q}(y|x_t, x_{t+1}) = \hat{q}(x_{t+1}|x_t, y)\frac{\hat{q}(y|x_t)}{\hat{q}(x_{t+1}|x_t)}$$

$$= \hat{q}(x_{t+1}|x_t)\frac{\hat{q}(y|x_t)}{\hat{q}(x_{t+1}|x_t)} = \hat{q}(y|x_t)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1}, y)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1}, y) = \frac{\hat{q}(x_t, x_{t+1}, y)}{\hat{q}(x_{t+1}, y)}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1}, y) = \frac{\hat{q}(x_t, x_{t+1}, y)}{\hat{q}(x_{t+1}, y)} = \frac{\hat{q}(x_t, x_{t+1}, y)}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1}, y) = \frac{\hat{q}(x_t, x_{t+1}, y)}{\hat{q}(x_{t+1}, y)} = \frac{\hat{q}(x_t, x_{t+1}, y)}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t, x_{t+1})\hat{q}(x_{t+1})}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1},y) = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(x_{t+1},y)} = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})\hat{q}(x_{t+1})}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} = \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})}{\hat{q}(y|x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1},y) = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(x_{t+1},y)} = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})\hat{q}(x_{t+1})}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} = \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})}{\hat{q}(y|x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t)}{\hat{q}(y|x_{t+1})}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1},y) = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(x_{t+1},y)} = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})\hat{q}(x_{t+1})}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} = \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})}{\hat{q}(y|x_{t+1})}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t)}{\hat{q}(y|x_{t+1})} = \frac{q(x_t|x_{t+1})\hat{q}(y|x_t)}{\hat{q}(y|x_{t+1})} \longleftarrow \text{Datailed proof in paper}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Now derive the conditional reverse process:

$$\hat{q}(x_t|x_{t+1},y) = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(x_{t+1},y)} = \frac{\hat{q}(x_t,x_{t+1},y)}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} \quad \cdots \text{ (By Bayes Rule)}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})\hat{q}(x_{t+1})}{\hat{q}(y|x_{t+1})\hat{q}(x_{t+1})} = \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t,x_{t+1})}{\hat{q}(y|x_{t+1})}$$

$$= \frac{\hat{q}(x_t|x_{t+1})\hat{q}(y|x_t)}{\hat{q}(y|x_{t+1})} = \frac{q(x_t|x_{t+1})\hat{q}(y|x_t)}{\hat{q}(y|x_{t+1})} = Zq(x_t|x_{t+1})\hat{q}(y|x_t)$$

$\hat{q}(y|x_{t+1})$: Constant

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Parameterize:

$$\hat{q}(x_t|x_{t+1}, y) = Zq(x_t|x_{t+1})\hat{q}(y|x_t)$$

$$p_{\theta,\phi}(x_t|x_{t+1}, y) = Zp_\theta(x_t|x_{t+1})p_\phi(y|x_t), where\ \phi\text{: Classifier}$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Recall:

$$p_\theta(x_t|x_{t+1}) = \mathcal{N}(\mu, \sum)$$

$$\log p_\theta(x_t|x_{t+1}) = -\frac{1}{2}(x_t - \mu)^T \Sigma^{-1}(x_t - \mu) + C$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu)^T \Sigma^{-1}(x_t - \mu) + \log p_\phi(y|x_t) + C$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu)^T \Sigma^{-1}(x_t - \mu) + \log p_\phi(y|x_t) + C$$

$$\approx -\frac{1}{2}(x_t - \mu)^T \Sigma^{-1}(x_t - \mu) + \log p_\phi(y|x_t)\Big|_{x_t=\mu} + (x_t - \mu)\nabla_{x_t} \log p_\phi(y|x_t)\Big|_{x_t=\mu} + C$$

Talyor expansion around $x_t = \mu$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t-\mu)^T\Sigma^{-1}(x_t-\mu) + \log p_\phi(y|x_t) + C$$

$$\approx -\frac{1}{2}(x_t-\mu)^T\Sigma^{-1}(x_t-\mu) + \log p_\phi(y|x_t)\Big|_{x_t=\mu} + (x_t-\mu)\nabla_{x_t}\log p_\phi(y|x_t)\Big|_{x_t=\mu} + C$$

$$= -\frac{1}{2}(x_t-\mu)^T\Sigma^{-1}(x_t-\mu) + (x_t-\mu)g + C$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t-\mu)^T\Sigma^{-1}(x_t-\mu) + \log p_\phi(y|x_t) + C$$

$$\approx -\frac{1}{2}(x_t-\mu)^T\Sigma^{-1}(x_t-\mu) + \log p_\phi(y|x_t)\Big|_{x_t=\mu} + (x_t-\mu)\nabla_{x_t}\log p_\phi(y|x_t)\Big|_{x_t=\mu} + C$$

$$= -\frac{1}{2}(x_t-\mu)^T\Sigma^{-1}(x_t-\mu) + (x_t-\mu)g + C$$

$$= -\frac{1}{2}(x_t-\mu-\Sigma g)^T\Sigma^{-1}(x_t-\mu-\Sigma g) + \frac{1}{2}g^T\Sigma g + C$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + \frac{1}{2} g^T \Sigma g + C$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + \frac{1}{2} g^T \Sigma g + C$$

$$= -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + C$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + \frac{1}{2} g^T \Sigma g + C$$

$$= -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + C$$

$$= \log p(x_t) + C, x_t \sim \mathcal{N}(\mu + \Sigma g, \Sigma) \longleftarrow \mathcal{N}(x|\mu, \Sigma) = \frac{1}{C}\exp(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu))$$

Multivariate Gaussian log likelihood

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + \frac{1}{2} g^T \Sigma g + C$$

$$= -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + C$$

$$= \log p(x_t) + C, x_t \sim \mathcal{N}(\mu + \Sigma g, \Sigma)$$

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Classifier Guidance Objective:

$$\log Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) = -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + \frac{1}{2} g^T \Sigma g + C$$

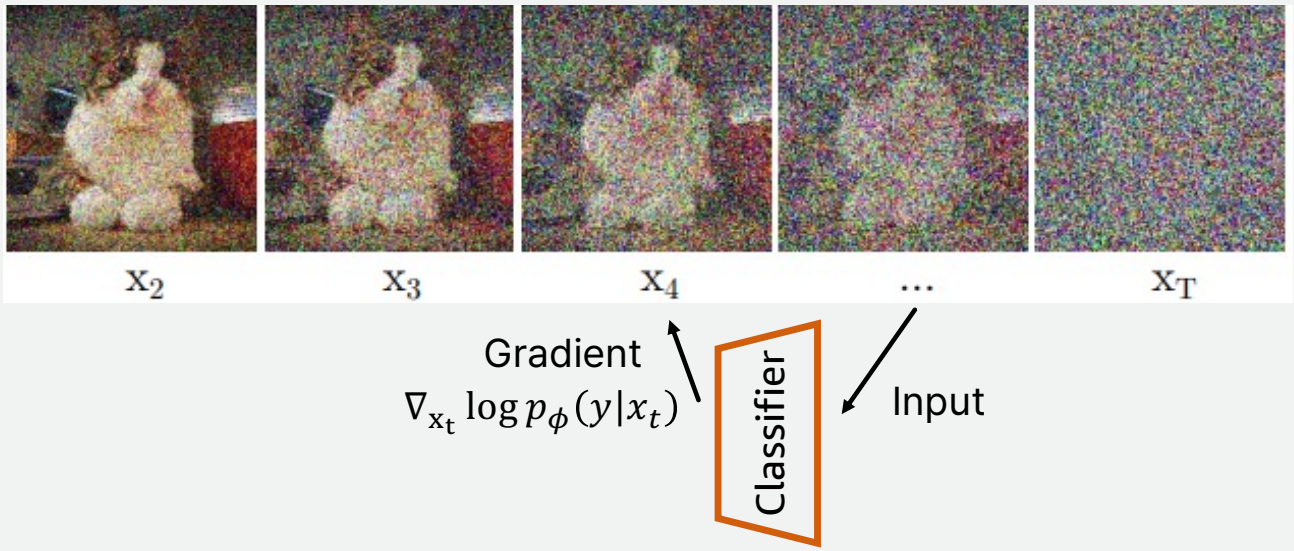
$$= -\frac{1}{2}(x_t - \mu - \Sigma g)^T \Sigma^{-1}(x_t - \mu - \Sigma g) + C$$

$$= \log p(x_t) + C, x_t \sim \mathcal{N}(\mu + \Sigma g, \Sigma)$$

Conditional generation is essentially a mean shift guided by the classifier's gradient!

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance



Conditional generation is essentially a mean shift guided by the classifier's gradient!

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $p_\phi(y|x_t)$, and gradient scale s.

Input: class label y, gradient scale s
$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$
for all t from T to 1 **do**
  $\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$
  $x_{t-1} \leftarrow$ sample from $\mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log p_\phi(y|x_t), \Sigma)$
end for
return x_0

Scaling factor

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $p_\phi(y|x_t)$, and gradient scale s.

Input: class label y, gradient scale s
$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$
for all t from T to 1 **do**
  $\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$
  $x_{t-1} \leftarrow$ sample from $\mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log p_\phi(y|x_t), \Sigma)$
end for
return x_0

Scaling factor

Algorithm 2 Classifier guided DDIM sampling, given a diffusion model $\epsilon_\theta(x_t)$, classifier $p_\phi(y|x_t)$, and gradient scale s.

Input: class label y, gradient scale s
$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$
for all t from T to 1 **do**
  $\hat{\epsilon} \leftarrow \epsilon_\theta(x_t) - \sqrt{1-\bar{\alpha}_t}\, \nabla_{x_t} \log p_\phi(y|x_t)$
  $x_{t-1} \leftarrow \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1-\bar{\alpha}_{t-1}}\hat{\epsilon}$
end for
return x_0

Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier Guidance





Dhariwal et al. Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021

Classifier-Free Guidance

Classifier Guidance has several issues:

1. We should train a classifier for our desired classes
2. This can be regarded as an exploit to artificially improve FID, IS scores
   - a form of adversarial attack

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda)$$

$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2$ (log signal-to-noise ratio)

$\alpha_\lambda :=$ signal scaling factor

$\sigma_\lambda :=$ noise scaling factor

$q(z_\lambda|x) = \mathcal{N}(\alpha_\lambda x, \sigma_\lambda^2 I)$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2$ (log signal-to-noise ratio)

$\alpha_\lambda :=$ signal scaling factor

$\sigma_\lambda :=$ noise scaling factor

$q(z_\lambda|x) = \mathcal{N}(\alpha_\lambda x, \sigma_\lambda^2 I)$

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda) \approx -\sigma_\lambda \nabla_{z_\lambda}[\log p(z_\lambda|c) + w \log p_\theta(c|z_\lambda)]$$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2$ (log signal-to-noise ratio)

$\alpha_\lambda :=$ signal scaling factor

$\sigma_\lambda :=$ noise scaling factor

$q(z_\lambda|x) = \mathcal{N}(\alpha_\lambda x, \sigma_\lambda^2 I)$

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda) \approx -\sigma_\lambda \nabla_{z_\lambda}[\log p(z_\lambda|c) + w \log p_\theta(c|z_\lambda)]$$

$$\epsilon_\theta(z_\lambda, c) \approx -\sigma_\lambda \nabla_{z_\lambda} \log p(z_\lambda|c)$$

Diffusion Score

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2$ (log signal-to-noise ratio)
$\alpha_\lambda :=$ signal scaling factor
$\sigma_\lambda :=$ noise scaling factor
$q(z_\lambda|x) = \mathcal{N}(\alpha_\lambda x, \sigma_\lambda^2 I)$

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda) \approx -\sigma_\lambda \nabla_{z_\lambda}[\log p(z_\lambda|c) + w \log p_\theta(c|z_\lambda)]$$

Classifier-Free Guidance:

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2$ (log signal-to-noise ratio)

$\alpha_\lambda :=$ signal scaling factor

$\sigma_\lambda :=$ noise scaling factor

$q(z_\lambda|x) = \mathcal{N}(\alpha_\lambda x, \sigma_\lambda^2 I)$

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda) \approx -\sigma_\lambda \nabla_{z_\lambda}[\log p(z_\lambda|c) + w \log p_\theta(c|z_\lambda)]$$

Classifier-Free Guidance:

Instead of using a classifier, let's use a single model
to parameterize both the conditional and unconditional models!

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

$$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2 \text{ (log signal-to-noise ratio)}$$
$$\alpha_\lambda := \text{signal scaling factor}$$
$$\sigma_\lambda := \text{noise scaling factor}$$
$$q(z_\lambda|x) = \mathcal{N}\left(\alpha_\lambda x, \sigma_\lambda^2 I\right)$$

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda)$$

Classifier-Free Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w(\epsilon_\theta(z_\lambda, \emptyset) - \epsilon_\theta(z_\lambda, c))$$

$$= (1 + w)\epsilon_\theta(z_\lambda, c) - w\epsilon_\theta(z_\lambda, \emptyset)$$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

$\lambda := \log \alpha_\lambda^2 / \sigma_\lambda^2$ (log signal-to-noise ratio)
$\alpha_\lambda :=$ signal scaling factor
$\sigma_\lambda :=$ noise scaling factor
$q(z_\lambda|x) = \mathcal{N}(\alpha_\lambda x, \sigma_\lambda^2 I)$

Classifier Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda)$$

Classifier-Free Guidance:

$$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w(\epsilon_\theta(z_\lambda, \emptyset) - \epsilon_\theta(z_\lambda, c))$$ No classifier gradient

$$= (1 + w)\epsilon_\theta(z_\lambda, c) - w\epsilon_\theta(z_\lambda, \emptyset)$$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Implicit classifier $p^i(c|z_\lambda) \propto \frac{p(z_\lambda|c)}{p(z_\lambda)}$

Conditional generator

Unconditional generator

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Implicit classifier $p^i(c|z_\lambda) \propto \frac{p(z_\lambda|c)}{p(z_\lambda)}$ ⟶ We can build implicit classifier using both model.

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Implicit classifier $p^i(c|z_\lambda) \propto \frac{p(z_\lambda|c)}{p(z_\lambda)}$

If we had exact score $\epsilon^*(z_\lambda, c), \epsilon^*(z_\lambda)$: score of $p(z_\lambda|c), p(z_\lambda)$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Implicit classifier $p^i(c|z_\lambda) \propto \frac{p(z_\lambda|c)}{p(z_\lambda)}$

If we had exact score $\epsilon^*(z_\lambda, c), \epsilon^*(z_\lambda)$: score of $p(z_\lambda|c), p(z_\lambda)$

Then $\nabla_{z_\lambda} \log p^i(c|z_\lambda) = -\frac{1}{\sigma_\lambda}[\epsilon^*(z_\lambda, c) - \epsilon^*(z_\lambda)]$,

$\epsilon_\theta(z_\lambda, c) \approx -\sigma_\lambda \nabla_{z_\lambda} \log p(z_\lambda|c)$

Diffusion Score

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Implicit classifier $p^i(c|z_\lambda) \propto \frac{p(z_\lambda|c)}{p(z_\lambda)}$

If we had exact score $\epsilon^*(z_\lambda, c), \epsilon^*(z_\lambda)$: score of $p(z_\lambda|c), p(z_\lambda)$

Then $\nabla_{z_\lambda} \log p^i(c|z_\lambda) = -\frac{1}{\sigma_\lambda}[\epsilon^*(z_\lambda, c) - \epsilon^*(z_\lambda)]$,

$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda)$ Classifier guidance

$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w(\epsilon_\theta(z_\lambda, \emptyset) - \epsilon_\theta(z_\lambda, c))$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Implicit classifier $p^i(c|z_\lambda) \propto \frac{p(z_\lambda|c)}{p(z_\lambda)}$

If we had exact score $\epsilon^*(z_\lambda, c), \epsilon^*(z_\lambda)$: score of $p(z_\lambda|c), p(z_\lambda)$

Then $\nabla_{z_\lambda} \log p^i(c|z_\lambda) = -\frac{1}{\sigma_\lambda}[\epsilon^*(z_\lambda, c) - \epsilon^*(z_\lambda)]$,

$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w\sigma_\lambda \nabla_{z_\lambda} \log p_\theta(c|z_\lambda)$

$\tilde{\epsilon}_\theta(z_\lambda, c) = \epsilon_\theta(z_\lambda, c) - w(\epsilon_\theta(z_\lambda, \emptyset) - \epsilon_\theta(z_\lambda, c))$ Classifier-free guidance

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Algorithm 1 Joint training a diffusion model with classifier-free guidance

Require: p_{uncond}: probability of unconditional training

repeat
  $(\mathbf{x}, \mathbf{c}) \sim p(\mathbf{x}, \mathbf{c})$ ▷ Sample data with conditioning from the dataset
  $\mathbf{c} \leftarrow \varnothing$ with probability p_{uncond} ▷ Randomly discard conditioning to train unconditionally
  $\lambda \sim p(\lambda)$ ▷ Sample log SNR value
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\mathbf{z}_\lambda = \alpha_\lambda \mathbf{x} + \sigma_\lambda \boldsymbol{\epsilon}$ ▷ Corrupt data to the sampled log SNR value
  Take gradient step on $\nabla_\theta \left\| \boldsymbol{\epsilon}_\theta(\mathbf{z}_\lambda, \mathbf{c}) - \boldsymbol{\epsilon} \right\|^2$ ▷ Optimization of denoising model
until converged

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Classifier-Free Guidance

Algorithm 1 Joint training a diffusion model with classifier-free guidance

Require: p_{uncond}: probability of unconditional training

repeat
  $(\mathbf{x}, \mathbf{c}) \sim p(\mathbf{x}, \mathbf{c})$ ▷ Sample data with conditioning from the dataset
  $\mathbf{c} \leftarrow \varnothing$ with probability p_{uncond} ▷ Randomly discard conditioning to train unconditionally
  $\lambda \sim p(\lambda)$ ▷ Sample log SNR value
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  $\mathbf{z}_\lambda = \alpha_\lambda \mathbf{x} + \sigma_\lambda \boldsymbol{\epsilon}$ ▷ Corrupt data to the sampled log SNR value
  Take gradient step on $\nabla_\theta \left\| \boldsymbol{\epsilon}_\theta(\mathbf{z}_\lambda, \mathbf{c}) - \boldsymbol{\epsilon} \right\|^2$ ▷ Optimization of denoising model
until converged

Algorithm 2 Conditional sampling with classifier-free guidance

Require: w: guidance strength
Require: $\mathbf{c}$: conditioning information for conditional sampling
Require: $\lambda_1, \ldots, \lambda_T$: increasing log SNR sequence with $\lambda_1 = \lambda_{\min}$, $\lambda_T = \lambda_{\max}$

$\mathbf{z}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = 1, \ldots, T$ **do**
  ▷ Form the classifier-free guided score at log SNR λ_t
  $\tilde{\boldsymbol{\epsilon}}_t = (1 + w)\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, \mathbf{c}) - w\boldsymbol{\epsilon}_\theta(\mathbf{z}_t)$
  ▷ Sampling step (could be replaced by another sampler, e.g. DDIM)
  $\tilde{\mathbf{x}}_t = (\mathbf{z}_t - \sigma_{\lambda_t} \tilde{\boldsymbol{\epsilon}}_t)/\alpha_{\lambda_t}$
  $\mathbf{z}_{t+1} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}_{\lambda_{t+1}|\lambda_t}(\mathbf{z}_t, \tilde{\mathbf{x}}_t), (\tilde{\sigma}^2_{\lambda_{t+1}|\lambda_t})^{1-v} (\sigma^2_{\lambda_t|\lambda_{t+1}})^v)$ if $t < T$ else $\mathbf{z}_{t+1} = \tilde{\mathbf{x}}_t$
end for
return $\mathbf{z}_{T+1}$

Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

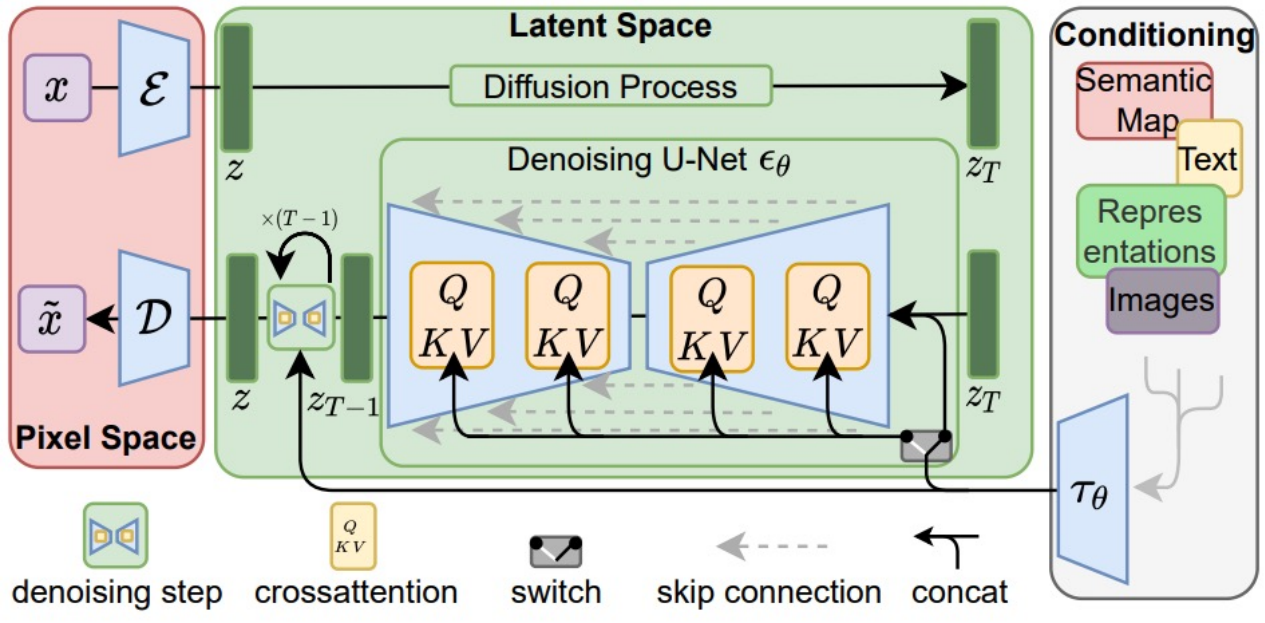
Classifier-Free Guidance



Ho et al. Classifier-Free Diffusion Guidance, NeurIPS 2021

Recent Conditioning Techniques

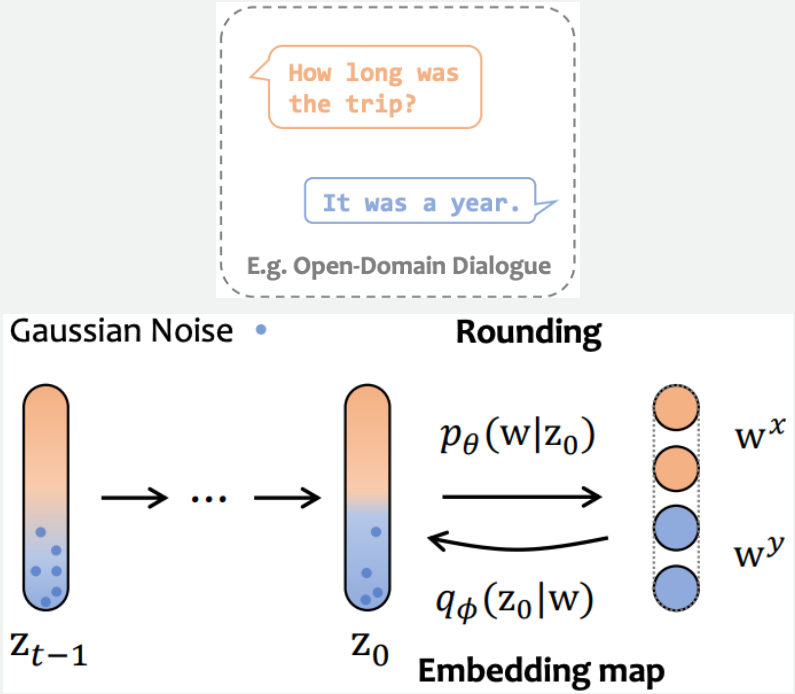
Cross-attention conditioning

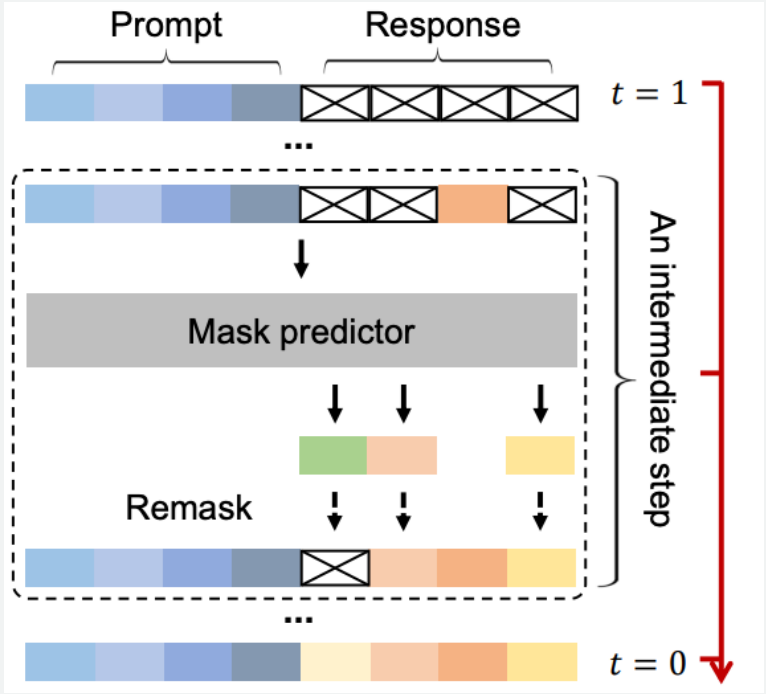


Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”, CVPR 2022

Recent Conditioning Techniques

In-context conditioning

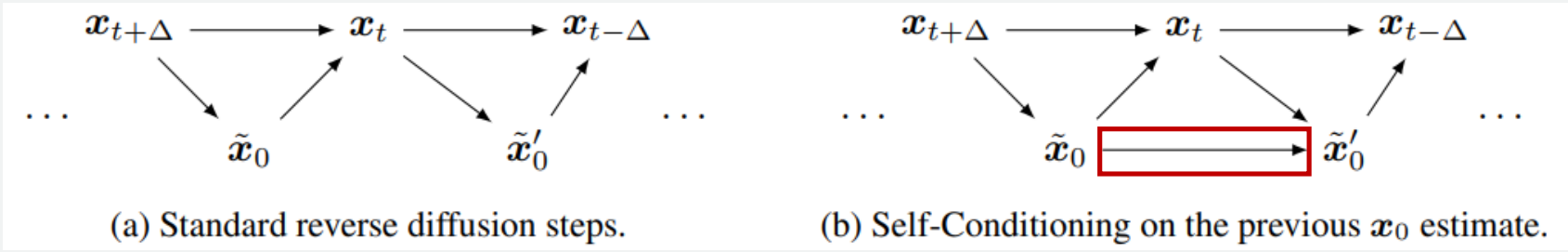




Gong et al. "DiffuSeq: Sequence to Sequence Text Generation with Diffusion Models", ICLR 2023
Nie et al. "Large Language Diffusion Models", Arxiv 2025

Recent Conditioning Techniques

Self-conditioning

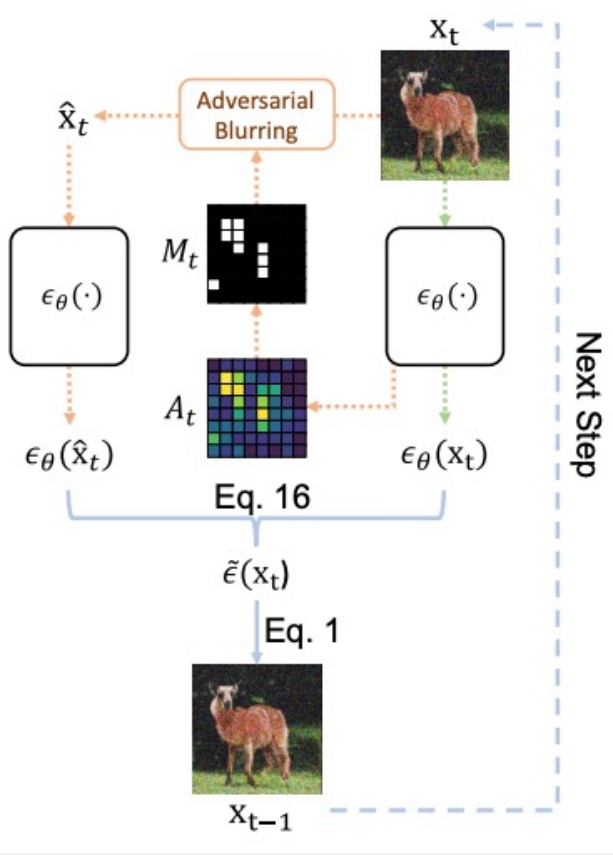


Concat x_t with $\tilde{x}_t$ and predict x_0

Chen et al. "Analog Bits: Generating Discrete Data using Diffusion Models with Self-Conditioning", ICLR 2023

Recent Conditioning Techniques

Self-conditioning



Hong et al. "Improving Sample Quality of Diffusion Models Using Self-Attention Guidance", ICCV 2023

Diffusion Distillation

What is (Diffusion) Distillation?

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Major barrier remains to practical adoption of diffusion models:

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Major barrier remains to practical adoption of diffusion models:

Sampling Speed

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Major barrier remains to practical adoption of diffusion models:

Sampling Speed

DDPM: about 1000 sampling steps

DDIM: about 100 sampling steps

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Major barrier remains to practical adoption of diffusion models:

Sampling Speed

DDPM: about 1000 sampling steps

DDIM: about 100 sampling steps

This means we need to call the model 100 to 1000 times to generate a single image

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Probability flow ODE [Diffusion model as ODE (Week5)]

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

$\alpha_t \coloneqq$ signal scaling factor
$\sigma_t \coloneqq$ noise scaling factor
$1 \geq \mathrm{t} > s \geq 0$: continuous time

Probability flow ODE [Diffusion model as ODE (Week5)]

$$dz_t = \left[f(z_t, t) - \frac{1}{2} g^2(t) \nabla_{\mathrm{z}} \log \hat{p}_\theta(z_t) \right] dt, \qquad \nabla_z \log \hat{p}_\theta(z_t) = \frac{\alpha_t \hat{x}_\theta(z_t) - z_t}{\sigma_t^2}$$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

$\alpha_t \coloneqq$ signal scaling factor
$\sigma_t \coloneqq$ noise scaling factor
$1 \geq \mathrm{t} > s \geq 0$: continuous time

Probability flow ODE [Diffusion model as ODE (Week5)]

$$dz_t = \left[f(z_t, t) - \frac{1}{2} g^2(t) \nabla_{\mathrm{z}} \log \hat{p}_\theta(z_t) \right] dt, \qquad \nabla_z \log \hat{p}_\theta(z_t) = \frac{\alpha_t \hat{x}_\theta(z_t) - z_t}{\sigma_t^2}$$

Sampler update rule for DDIM solving ODE

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

$\alpha_t \coloneqq$ signal scaling factor
$\sigma_t \coloneqq$ noise scaling factor
$1 \geq \mathrm{t} > s \geq 0$: continuous time

Probability flow ODE [Diffusion model as ODE (Week5)]

$$dz_t = \left[f(z_t, t) - \frac{1}{2} g^2(t) \nabla_z \log \hat{p}_\theta(z_t) \right] dt, \qquad \nabla_z \log \hat{p}_\theta(z_t) = \frac{\alpha_t \hat{x}_\theta(z_t) - z_t}{\sigma_t^2}$$

Sampler update rule for DDIM solving ODE

$$z_s = \alpha_s \hat{x}_\theta(z_t) + \sigma_s \frac{z_t - \alpha_t \hat{x}_\theta(z_t)}{\sigma_t} = e^{\frac{\lambda_t - \lambda_s}{2}} \left(\frac{\alpha_s}{\alpha_t} \right) z_t + \left(1 - e^{\frac{\lambda_t - \lambda_s}{2}} \right) \alpha_s \hat{x}_\theta(z_t)$$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Probability flow ODE [Diffusion model as ODE (Week5)]

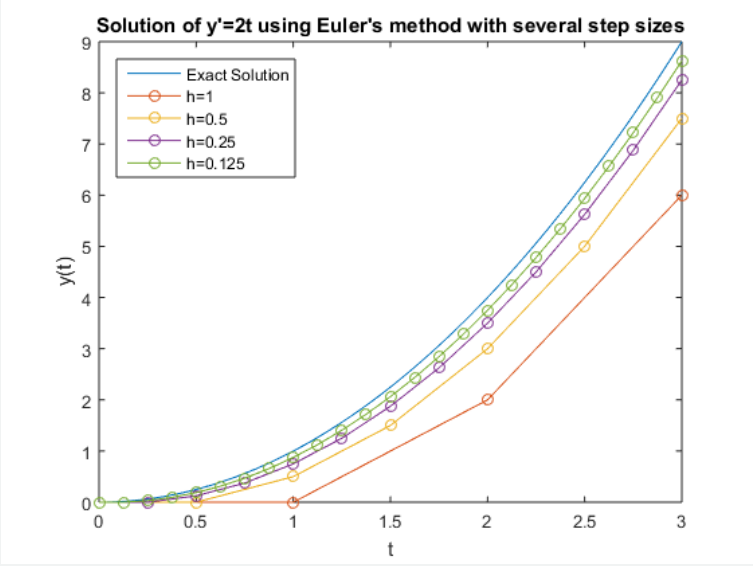
If the integration step count $N \to \infty$ and $\hat{x}_\theta(z_t)$ satiesfies *mild smoothness conditions*, the numerical error is guaranteed to vanish

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

Probability flow ODE [Diffusion model as ODE (Week5)]

If the integration step count $N \to \infty$ and $\hat{x}_\theta(z_t)$ satiesfies *mild smoothness conditions*, the numerical error is guaranteed to vanish



Salimans et al. "Progressive Distillation for Fast Sampling of Diffusion Models", ICLR 2022

What is (Diffusion) Distillation?

To generate samples fast, we need fast ODE solvers

Sampler	Number of steps	FID
Stochastic	1000	**13.35**
Euler	1000	16.5
RK4	1000	16.33
DDIM	1000	15.98
Stochastic	100	18.44
Euler	100	23.67
RK4	100	18.94
DDIM	100	**16.35**

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

What is (Diffusion) Distillation?

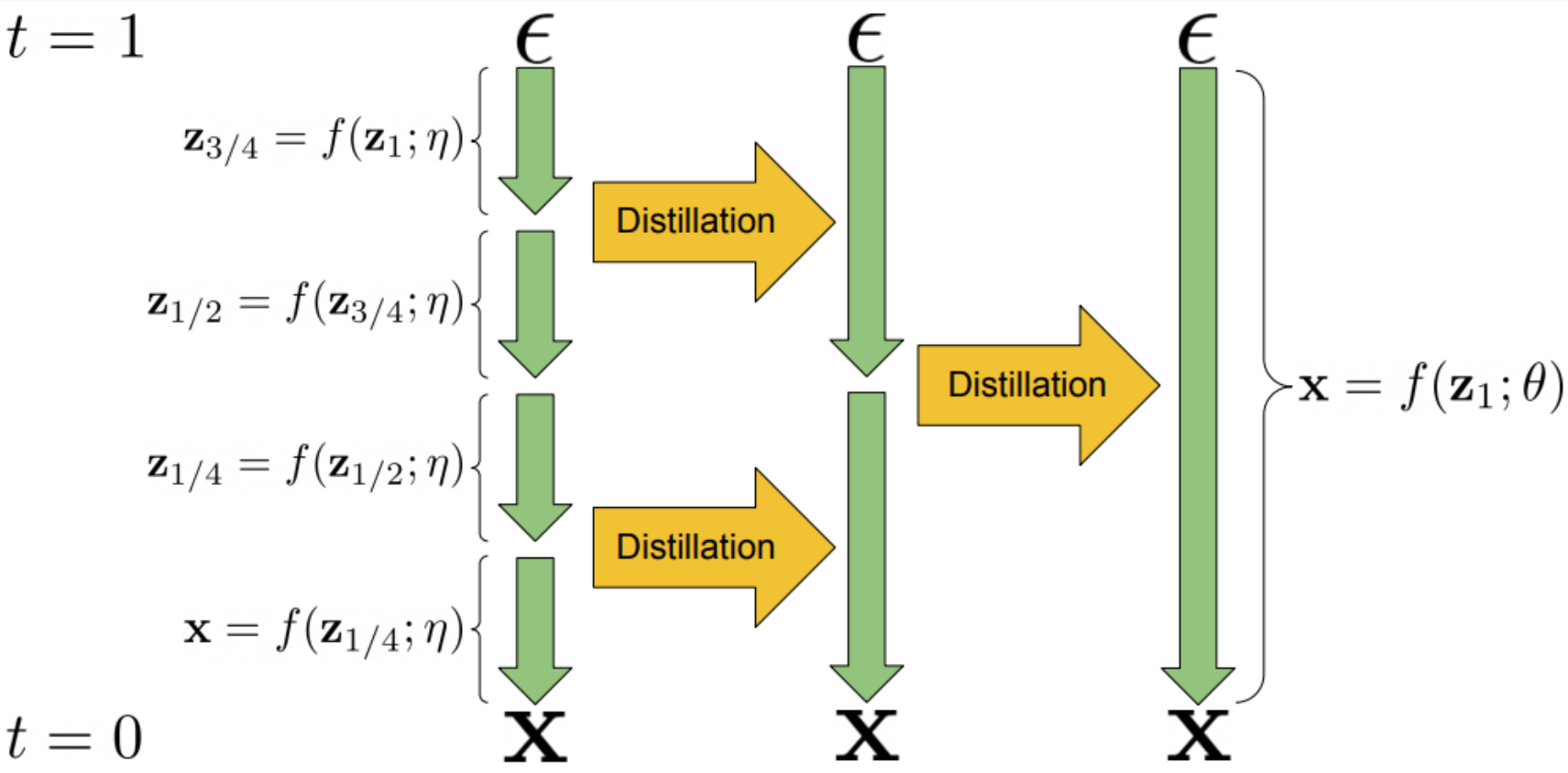
To generate samples fast, we need fast ODE solvers

Diffusion Distullation

ODE solver trained to amortizes the integration process of the teacher model.

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation



Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

If $\alpha_t \to 0$, small changes in the NN output are increasingly amplified

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

If $\alpha_t \to 0$, small changes in the NN output are increasingly amplified

This is not much of a problem in many-step diffusion, as the steps correct errors

Salimans et al. "Progressive Distillation for Fast Sampling of Diffusion Models", ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

If $\alpha_t \to 0$, small changes in the NN output are increasingly amplified

This is not much of a problem in many-step diffusion, as the steps correct errors

Suppose we distilled a 1-step diffusion model. Then…

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

If $\alpha_t \to 0$, small changes in the NN output are increasingly amplified

This is not much of a problem in many-step diffusion, as the steps correct errors

Suppose we distilled a 1-step diffusion model. Then…

$\hat{\epsilon}_\theta(z_t)$ no longer predicts x

Salimans et al. "Progressive Distillation for Fast Sampling of Diffusion Models", ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

x-prediction: predicting x directly

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

x-prediction: predicting x directly

x, ϵ-prediction: predicting x, ϵ, merging them via $\hat{x} = \sigma_t^2 \, \tilde{x}_\theta(z_t) + \alpha_t(z_t - \sigma_t \tilde{\epsilon}_\theta(z_t))$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

x-prediction: predicting x directly

x, ϵ-prediction: predicting x, ϵ, merging them via $\hat{x} = \sigma_t^2 \, \tilde{x}_\theta(z_t) + \alpha_t(z_t - \sigma_t \tilde{\epsilon}_\theta(z_t))$

v-prediction: $\hat{v}_\theta(z_t) = \alpha_t \epsilon_t - \sigma_t x$, $\hat{x} = \alpha_t z_t - \sigma_t \hat{v}_\theta(z_t)$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Original model loss: $L_\theta = \frac{\alpha_t^2}{\sigma_t^2} \|x - \hat{x}_\theta(z_t)\|_2^2$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Original model loss: $L_\theta = \frac{\alpha_t^2}{\sigma_t^2} \|x - \hat{x}_\theta(z_t)\|_2^2$

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Original model loss: $L_\theta = \frac{\alpha_t^2}{\sigma_t^2} \|x - \hat{x}_\theta(z_t)\|_2^2$

If $\alpha_t \to 0$, loss becomes zero

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Original model prediction: $\hat{x}_\theta(z_t) = \frac{1}{\alpha_t}(z_t - \sigma_t \hat{\epsilon}_\theta(z_t))$

Original model loss: $L_\theta = \frac{\alpha_t^2}{\sigma_t^2} \|x - \hat{x}_\theta(z_t)\|_2^2$

$L_\theta = \max\left(\frac{\alpha_t^2}{\sigma_t^2}, 1\right) \|x - \hat{x}_\theta(z_t)\|_2^2$; Truncated SNR weighting

$L_\theta = (\frac{\alpha_t^2}{\sigma_t^2} + 1) \|x - \hat{x}_\theta(z_t)\|_2^2$; SNR+1 weighting

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation

Algorithm 1 Standard diffusion training

Require: Model $\hat{\mathbf{x}}_\theta(\mathbf{z}_t)$ to be trained
Require: Data set $\mathcal{D}$
Require: Loss weight function $w()$

while not converged **do**
  $\mathbf{x} \sim \mathcal{D}$ ▷ Sample data
  $t \sim U[0, 1]$ ▷ Sample time
  $\epsilon \sim N(0, I)$ ▷ Sample noise
  $\mathbf{z}_t = \alpha_t \mathbf{x} + \sigma_t \epsilon$ ▷ Add noise to data

  $\tilde{\mathbf{x}} = \mathbf{x}$ ▷ Clean data is target for $\hat{\mathbf{x}}$
  $\lambda_t = \log[\alpha_t^2 / \sigma_t^2]$ ▷ log-SNR
  $L_\theta = w(\lambda_t) \|\tilde{\mathbf{x}} - \hat{\mathbf{x}}_\theta(\mathbf{z}_t)\|_2^2$ ▷ Loss
  $\theta \leftarrow \theta - \gamma \nabla_\theta L_\theta$ ▷ Optimization
end while

Algorithm 2 Progressive distillation

Require: Trained teacher model $\hat{\mathbf{x}}_\eta(\mathbf{z}_t)$
Require: Data set $\mathcal{D}$
Require: Loss weight function $w()$
Require: Student sampling steps N
for K iterations **do**
  $\theta \leftarrow \eta$ ▷ Init student from teacher
  while not converged **do**
    $\mathbf{x} \sim \mathcal{D}$
    $t = i/N, \;\; i \sim Cat[1, 2, \ldots, N]$
    $\epsilon \sim N(0, I)$
    $\mathbf{z}_t = \alpha_t \mathbf{x} + \sigma_t \epsilon$
    `# 2 steps of DDIM with teacher`
    $t' = t - 0.5/N, \;\; t'' = t - 1/N$
    $\mathbf{z}_{t'} = \alpha_{t'} \hat{\mathbf{x}}_\eta(\mathbf{z}_t) + \frac{\sigma_{t'}}{\sigma_t}(\mathbf{z}_t - \alpha_t \hat{\mathbf{x}}_\eta(\mathbf{z}_t))$
    $\mathbf{z}_{t''} = \alpha_{t''} \hat{\mathbf{x}}_\eta(\mathbf{z}_{t'}) + \frac{\sigma_{t''}}{\sigma_{t'}}(\mathbf{z}_{t'} - \alpha_{t'} \hat{\mathbf{x}}_\eta(\mathbf{z}_{t'}))$
    $\tilde{\mathbf{x}} = \frac{\mathbf{z}_{t''} - (\sigma_{t''}/\sigma_t)\mathbf{z}_t}{\alpha_{t''} - (\sigma_{t''}/\sigma_t)\alpha_t}$ ▷ Teacher $\hat{\mathbf{x}}$ target
    $\lambda_t = \log[\alpha_t^2 / \sigma_t^2]$
    $L_\theta = w(\lambda_t) \|\tilde{\mathbf{x}} - \hat{\mathbf{x}}_\theta(\mathbf{z}_t)\|_2^2$
    $\theta \leftarrow \theta - \gamma \nabla_\theta L_\theta$
  end while
  $\eta \leftarrow \theta$ ▷ Student becomes next teacher
  $N \leftarrow N/2$ ▷ Halve number of sampling steps
end for

Salimans et al. "Progressive Distillation for Fast Sampling of Diffusion Models", ICLR 2022

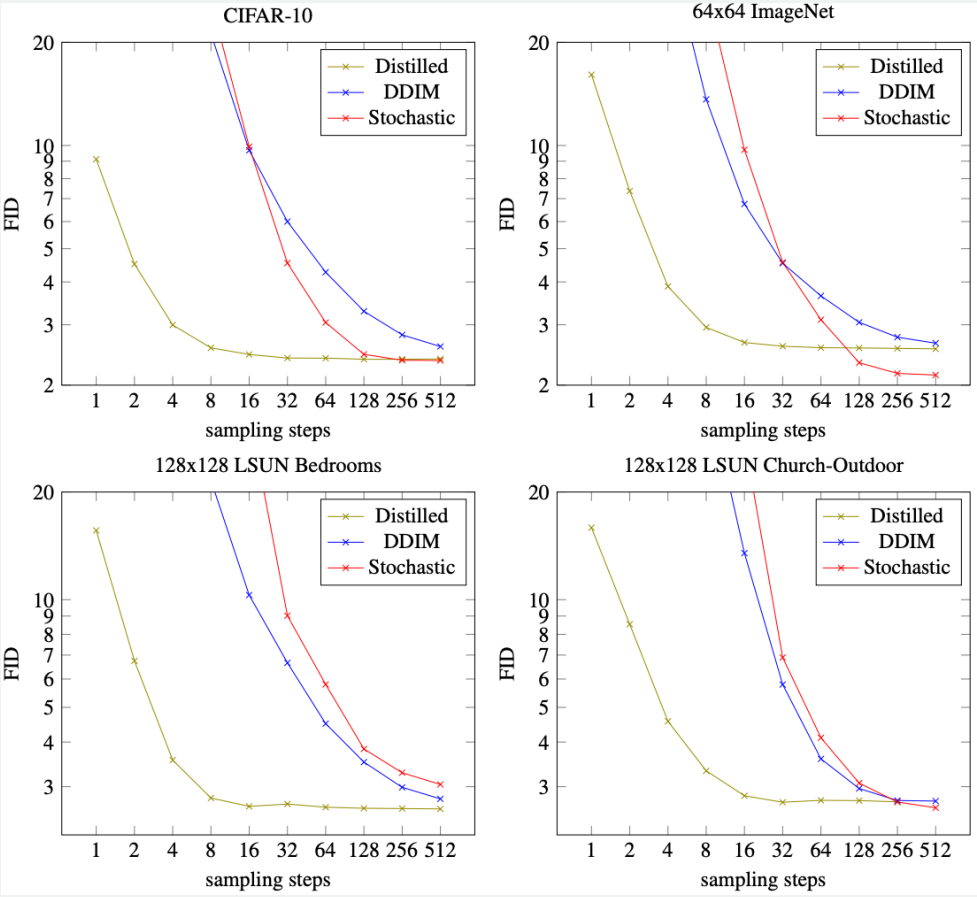
Diffusion Distillation



(a) 256 sampling steps (b) 4 sampling steps (c) 1 sampling step

Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Diffusion Distillation



Salimans et al. “Progressive Distillation for Fast Sampling of Diffusion Models”, ICLR 2022

Consistency Distillation

Consistency Models

Song et al. “Consistency Models”, ICML 2022

Consistency Distillation

Consistency Models

Consistency function: $\forall x_t$ along the PF-ODE trajectories, return x_0

Song et al. “Consistency Models”, ICML 2022

Consistency Distillation

Consistency Models

Consistency function: $\forall x_t$ along the PF-ODE trajectories, return x_0

- Self-consistency: $\forall x_t, f_\theta(x_t, t) = f_\theta(x'_t, t'), \forall t, t' \in [0, T]$

- Boundary condition: Given $x_0, f_\theta(x_0, 0) = x_0$

Song et al. “Consistency Models”, ICML 2022

Consistency Distillation

Consistency Models

Consistency function: $\forall x_t$ along the PF-ODE trajectories, return x_0

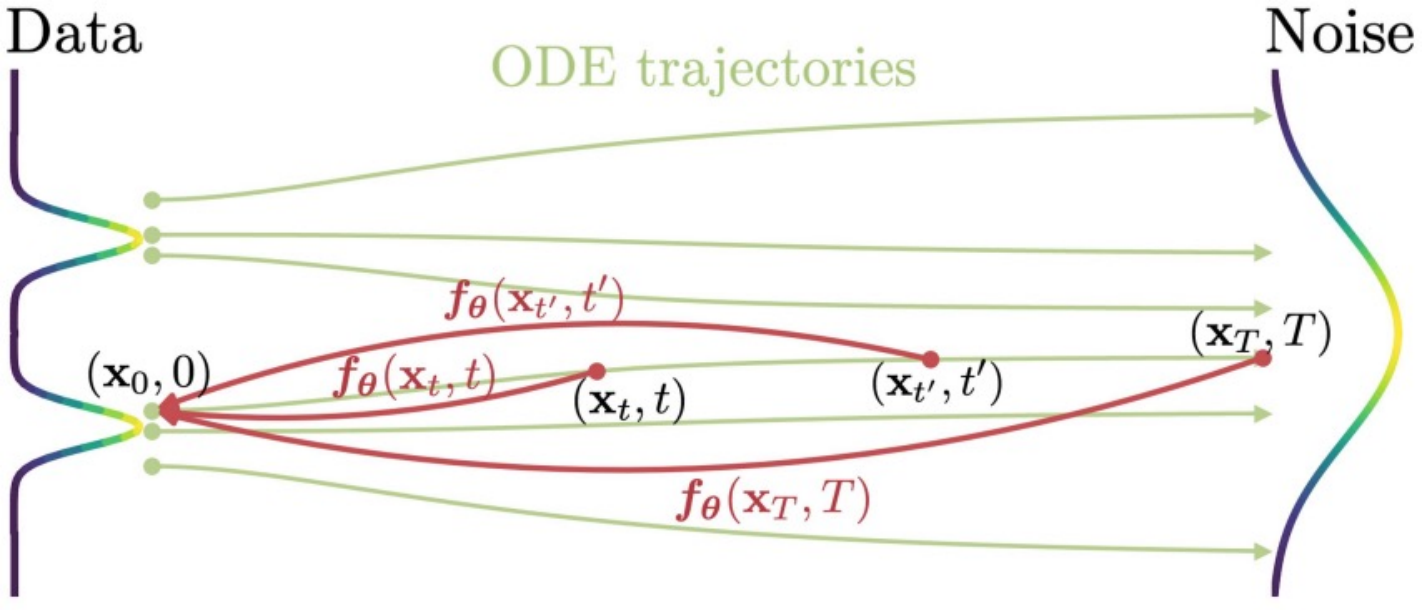
- Self-consistency: $\forall x_t, f_\theta(x_t, t) = f_\theta(x'_t, t'), \forall t, t' \in [0, T]$
- Boundary condition: Given $x_0, f_\theta(x_0, 0) = x_0$

$$f_\theta(x_t, t) = c_{skip}(t)x_t + c_{out}F_\theta(x_t, t), where\ c_{skip}(0) = 1\ and\ c_{out}(0) = 0$$

Song et al. “Consistency Models”, ICML 2022

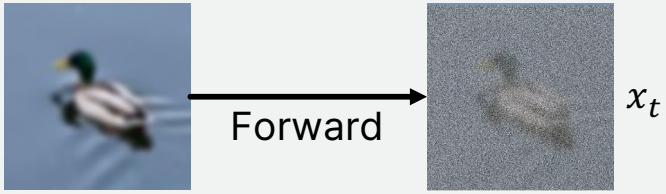
Consistency Distillation

Consistency Models



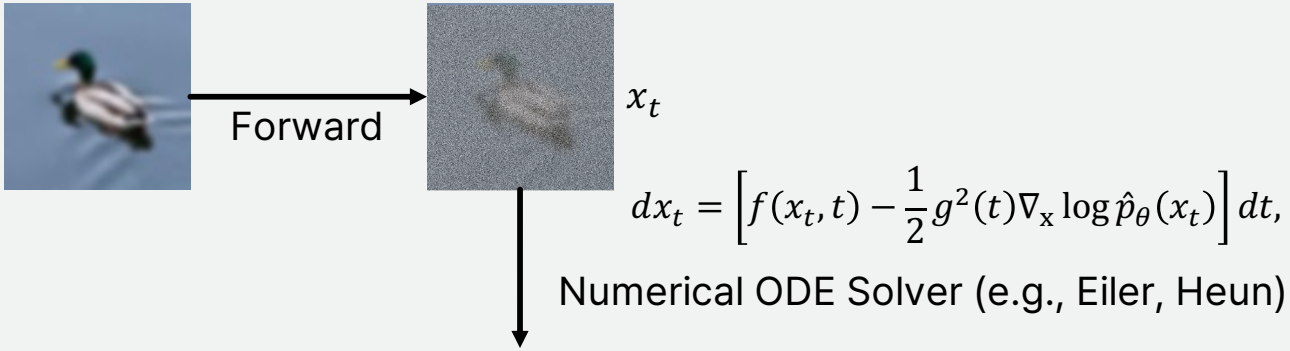
Song et al. “Consistency Models”, ICML 2022

Consistency Distillation



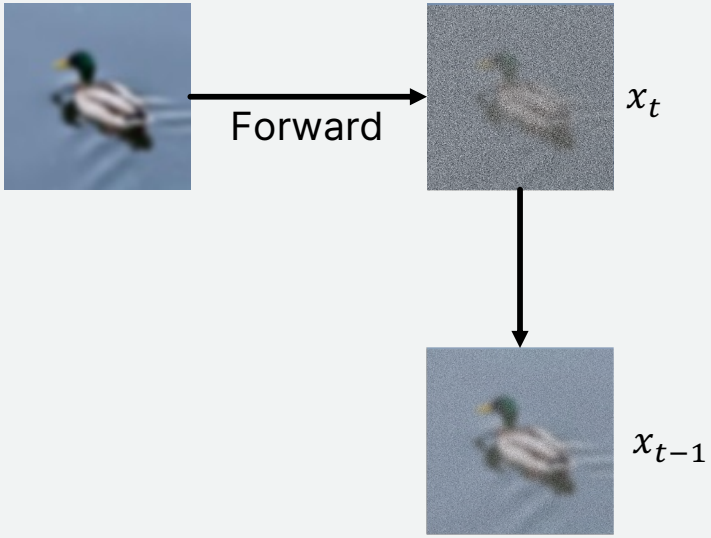
Song et al. “Consistency Models”, ICML 2022

Consistency Distillation



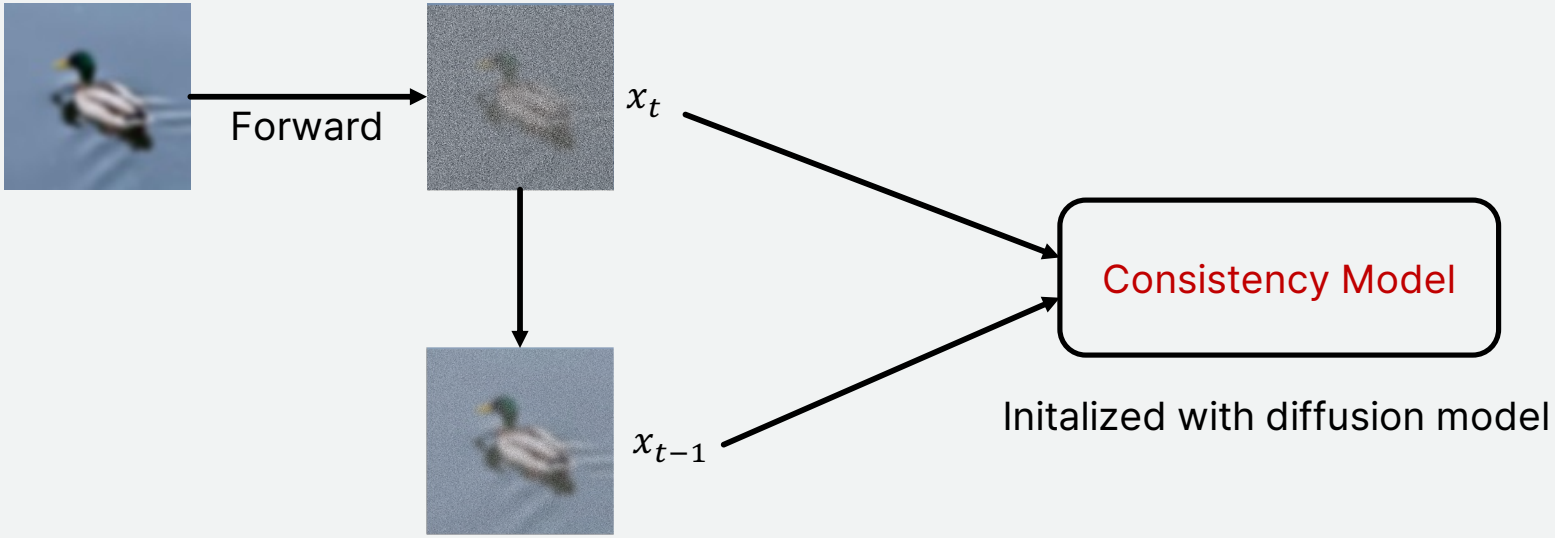
Song et al. “Consistency Models”, ICML 2022

Consistency Distillation



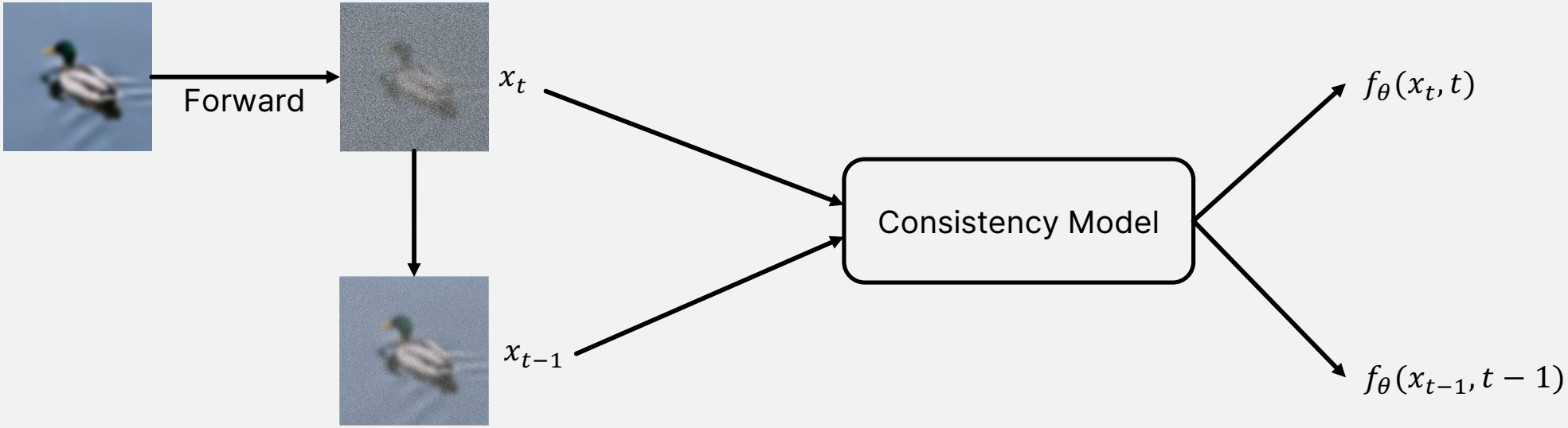
Song et al. “Consistency Models”, ICML 2022

Consistency Distillation



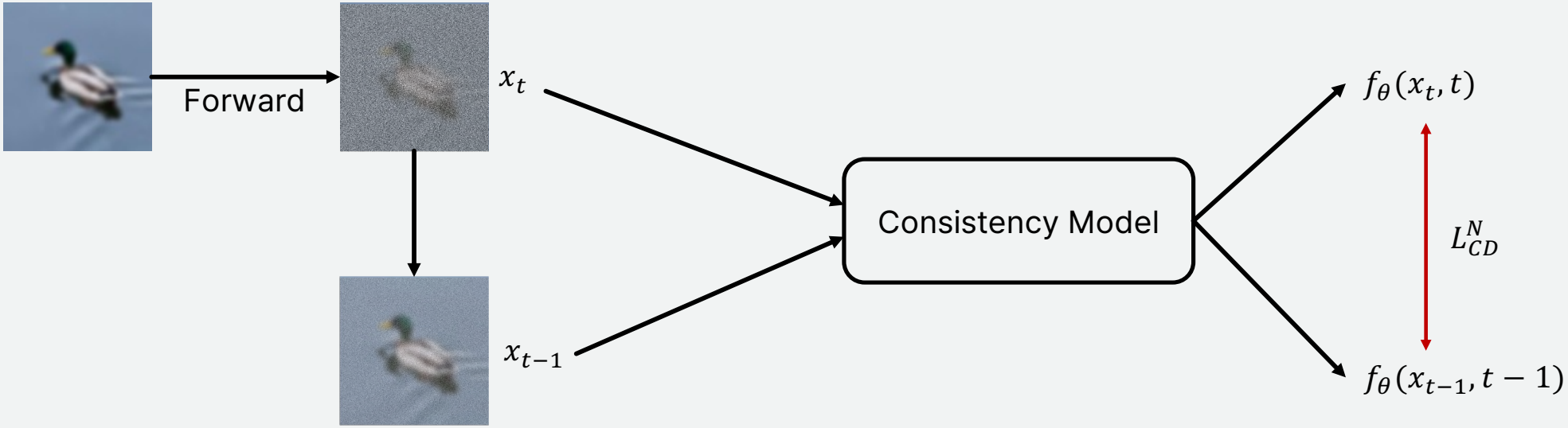
Song et al. “Consistency Models”, ICML 2022

Consistency Distillation



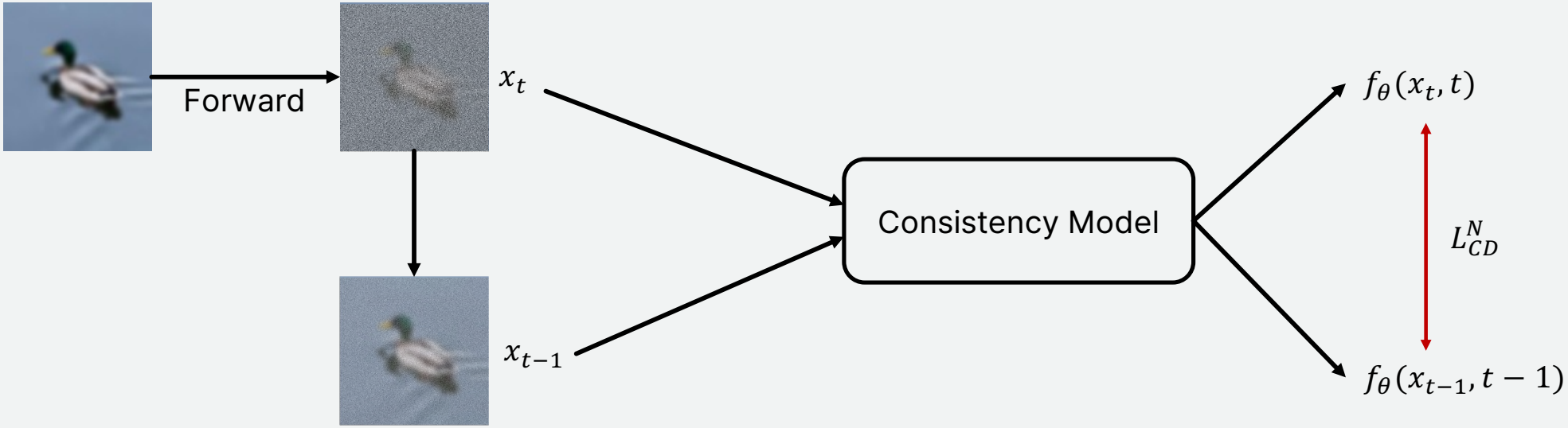
Song et al. “Consistency Models”, ICML 2022

Consistency Distillation



Song et al. “Consistency Models”, ICML 2022

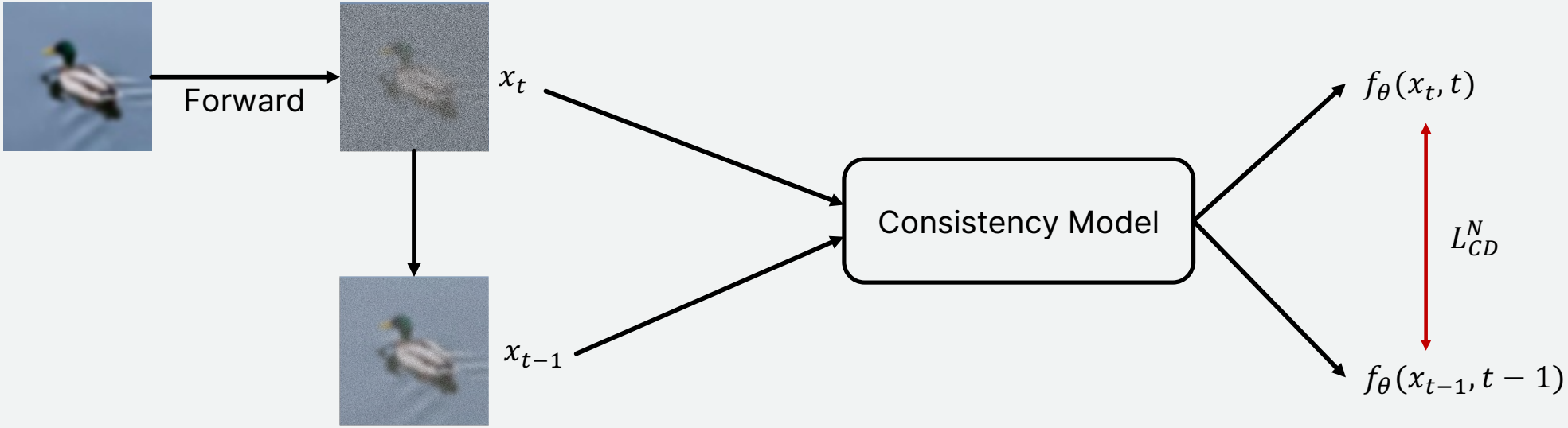
Consistency Distillation



$$L_{CD}^N(\theta, \theta^-; \phi) := \mathbb{E}[\lambda(t) d(f_\theta(x_t, t), f_{\theta^-}(x_{t-1}, t-1))]$$

Song et al. "Consistency Models", ICML 2022

Consistency Distillation

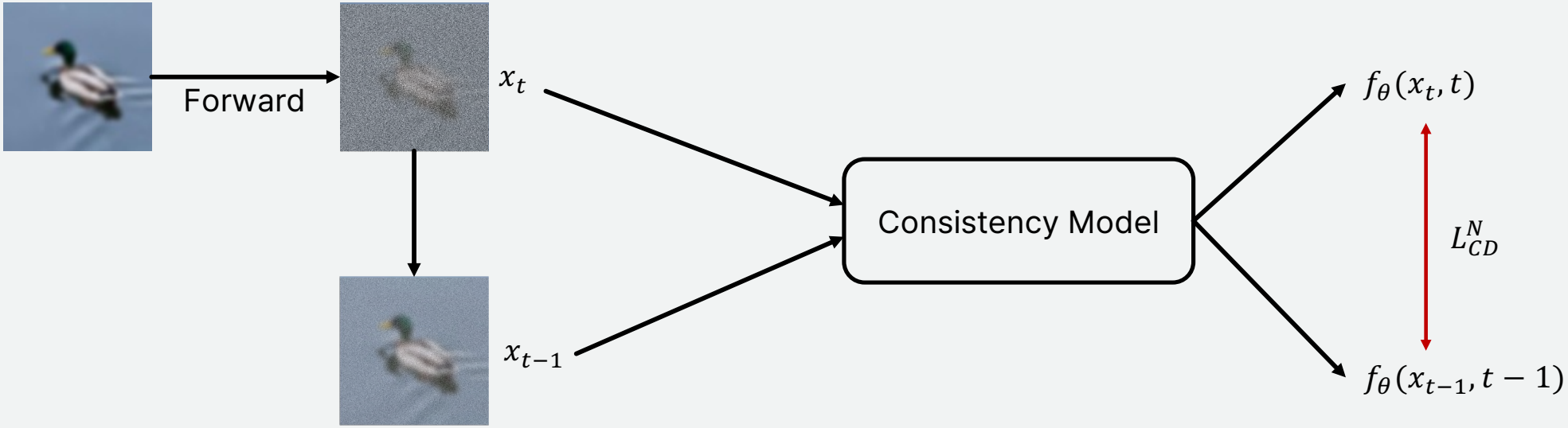


$$L_{CD}^N(\theta, \theta^-; \phi) := \mathbb{E}[\lambda(t) d(f_\theta(x_t, t), f_{\theta^-}(x_{t-1}, t-1))]$$

Distance (e.g., $l^2 - distance, LPIPS$ ...)

Song et al. "Consistency Models", ICML 2022

Consistency Distillation



$$L_{CD}^N(\theta, \theta^-; \phi) := \mathbb{E}[\lambda(t) d(f_\theta(x_t, t), f_{\theta^-}(x_{t-1}, t-1))]$$

Target model with EMA update ($\theta^- \leftarrow \mu\theta^- + (1-\mu)\theta$)

Song et al. "Consistency Models", ICML 2022

Consistency Distillation

METHOD	NFE (↓)	FID (↓)	IS (↑)
Diffusion + Samplers			
DDIM (Song et al., 2020)	50	4.67	
DDIM (Song et al., 2020)	20	6.84	
DDIM (Song et al., 2020)	10	8.23	
DPM-solver-2 (Lu et al., 2022)	10	5.94	
DPM-solver-fast (Lu et al., 2022)	10	4.70	
3-DEIS (Zhang & Chen, 2022)	10	**4.17**	
Diffusion + Distillation			
Knowledge Distillation* (Luhman & Luhman, 2021)	1	9.36	
DFNO* (Zheng et al., 2022)	1	4.12	
1-Rectified Flow (+distill)* (Liu et al., 2022)	1	6.18	9.08
2-Rectified Flow (+distill)* (Liu et al., 2022)	1	4.85	9.01
3-Rectified Flow (+distill)* (Liu et al., 2022)	1	5.21	8.79
PD (Salimans & Ho, 2022)	1	8.34	8.69
CD	1	**3.55**	**9.48**
PD (Salimans & Ho, 2022)	2	5.58	9.05
CD	2	**2.93**	**9.75**

Song et al. “Consistency Models”, ICML 2022

Preview

Future Weeks

Now we begin diffusion language models!

Week 8-9: Diffuson Language Models (DLMs)

Week 10: Semi-AR, Block Diffusion…

Week 11: Scaling DLMs

Song et al. “Consistency Models”, ICML 2022

Thank you!